

UNIVERSIDAD DEL VALLE DE GUATEMALA

Inteligencia Artificial

Proyecto 2

Algoritmos de Búsqueda

Link al repositorio:

<https://github.com/Albu231311/Proyecto2-IA>

Fernando Ruiz - 23065

Kevin Villagrán - 23584

Nery Molina - 23218

Carlos Alburez - 231311

Guatemala, 2026

Índice

1. Introducción	2
2. Generación aleatoria de laberintos	2
2.1. Representación del laberinto	2
2.2. Algoritmo de Kruskal	2
2.3. Algoritmo de Prim	2
2.4. Resultados de generación	3
3. Solución de un laberinto aleatorio	4
3.1. Algoritmos de búsqueda implementados	4
3.2. Resultados del laberinto 60×80	4
4. Comparación de algoritmos de búsqueda	5
4.1. Resumen promedio	6
4.2. Escenarios de ejemplo	6
4.3. Tabla detallada de escenarios	8
5. Descripción técnica de la implementación	9
6. Conclusiones	9

1. Introducción

El presente proyecto implementa y compara algoritmos de generación y resolución de laberintos desde un enfoque de Inteligencia Artificial. El trabajo se divide en tres partes: la generación aleatoria de laberintos mediante algoritmos basados en árboles de expansión, la búsqueda de una ruta de salida en un laberinto de tamaño fijo y la comparación experimental de cuatro algoritmos de búsqueda sobre veinticinco escenarios distintos.

La implementación fue desarrollada en Python y utiliza `numpy` para la representación matricial, `matplotlib` para las visualizaciones y estructuras estándar como colas, pilas y colas de prioridad para los algoritmos de búsqueda. El repositorio contiene dos módulos principales: `maze_generator.py`, encargado de construir y visualizar laberintos, y `maze_solver.py`, encargado de resolverlos y comparar el desempeño de los algoritmos.

2. Generación aleatoria de laberintos

2.1. Representación del laberinto

El laberinto se modela mediante una matriz binaria de tamaño $(2M - 1) \times (2N - 1)$ para un grid lógico de $M \times N$ celdas. Las posiciones pares de la matriz representan celdas transitables y las posiciones intermedias representan paredes entre celdas adyacentes. En la implementación, el valor 1 representa un pasillo abierto y el valor 0 representa una pared.

La clase `Maze` almacena el número de filas y columnas, la matriz del laberinto y operaciones auxiliares para abrir paredes y obtener vecinos válidos. Esta representación permite convertir de manera directa un árbol de expansión en un laberinto perfecto, es decir, un laberinto conectado y sin ciclos donde existe exactamente un camino entre dos celdas cualesquiera.

2.2. Algoritmo de Kruskal

El algoritmo de Kruskal genera el laberinto considerando cada celda como un conjunto independiente. Primero se enumeran todas las paredes entre celdas vecinas y se barajan aleatoriamente. Luego se procesa cada pared: si las dos celdas que separa pertenecen a componentes distintas, se elimina la pared y se unen ambos conjuntos.

Para evitar ciclos se utiliza una estructura *Union-Find*, con compresión de caminos y unión por rango. Esta estructura permite verificar eficientemente si dos celdas ya están conectadas. El resultado final es un árbol de expansión sobre todas las celdas del grid.

2.3. Algoritmo de Prim

El algoritmo de Prim implementado utiliza una versión aleatoria. Se selecciona una celda inicial y se mantiene una frontera de conexiones entre celdas visitadas y no visitadas. En

cada iteración se elige aleatoriamente una conexión de la frontera; si conduce a una celda nueva, se abre la pared correspondiente y se agregan sus vecinos no visitados a la frontera.

Al igual que Kruskal, Prim produce un laberinto perfecto. Sin embargo, su proceso de construcción es distinto: Kruskal une componentes dispersas, mientras que Prim expande una región conectada desde una celda inicial.

2.4. Resultados de generación

Para la visualización del Problema 1 se generaron laberintos de 30×40 celdas con semilla fija. Ambos algoritmos abrieron 1199 paredes, valor esperado para conectar 1200 celdas mediante un árbol de expansión. La Tabla 1 resume los resultados observados.

Cuadro 1: Resultados de generación de laberintos 30×40 .

Métrica	Kruskal	Prim
Paredes abiertas	1199	1199
Tiempo de generación (ms)	8.98	8.09
Celdas totales	1200	1200



Figura 1: Comparación estática de laberintos generados con Kruskal y Prim.

Además de la figura estática, el programa genera animaciones de construcción para cada algoritmo y una animación comparativa lado a lado. Los archivos producidos son `kruskal_animacion.gif`, `prim_animacion.gif` y `construccion_comparacion.gif`.

3. Solución de un laberinto aleatorio

Para el Problema 2 se generó un laberinto de 60×80 usando Kruskal con semilla fija. La entrada corresponde a la celda $A = (1, 1)$ en notación del enunciado, representada internamente como $(0, 0)$, y la salida corresponde a $B = (60, 80)$, representada internamente como $(59, 79)$.

3.1. Algoritmos de búsqueda implementados

BFS. La búsqueda en anchura utiliza una cola FIFO. Explora primero los nodos a menor profundidad, por lo que encuentra rutas óptimas cuando todos los movimientos tienen el mismo costo.

DFS. La búsqueda en profundidad utiliza una pila. Puede encontrar una ruta rápidamente si el orden de expansión se alinea con la salida, aunque en grafos generales no garantiza optimalidad. En este proyecto, al trabajar con laberintos perfectos, la ruta entre dos celdas es única.

UCS. La búsqueda de costo uniforme utiliza una cola de prioridad ordenada por costo acumulado. Como todos los movimientos tienen costo 1, su comportamiento en longitud de ruta coincide con BFS, aunque con mayor costo administrativo por el uso del heap.

A*. El algoritmo A* combina el costo acumulado $g(n)$ con una heurística $h(n)$. Se utilizó distancia Manhattan:

$$h(n) = |r_n - r_{meta}| + |c_n - c_{meta}|.$$

Esta heurística es admisible para movimientos ortogonales, por lo que orienta la búsqueda hacia la meta sin sobreestimar el costo restante.

3.2. Resultados del laberinto 60×80

La Tabla 2 muestra el desempeño de los cuatro algoritmos sobre el laberinto 60×80 . Todos encontraron una ruta de longitud 387 celdas. DFS exploró menos nodos en este escenario particular, mientras que BFS y UCS exploraron la misma cantidad debido a que el costo de cada movimiento es uniforme.

Cuadro 2: Resultados de búsqueda en el laberinto 60×80 .

Algoritmo	Longitud del camino	Nodos explorados	Tiempo (ms)
BFS	387	4752	8.74
DFS	387	1317	2.66
UCS	387	4752	12.28
A*	387	4534	12.73

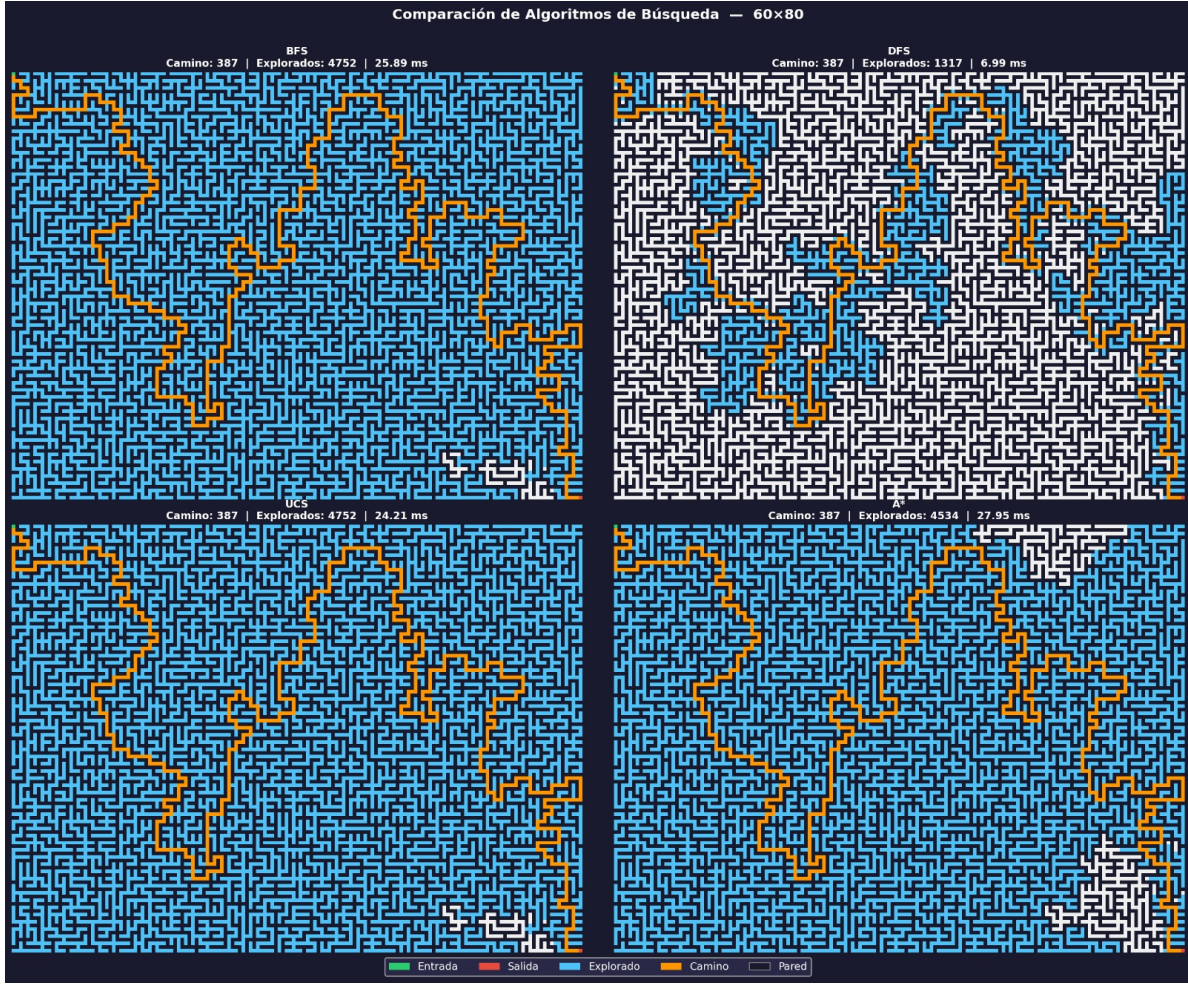


Figura 2: Celdas exploradas y camino encontrado por BFS, DFS, UCS y A* en el laberinto 60×80 .

Las animaciones individuales generadas para este problema son `bfs_solucion.gif`, `dfs_solucion.gif`, `ucs_solucion.gif` y `astar_solucion.gif`. En ellas se observa la región explorada y posteriormente el trazado del camino encontrado.

4. Comparación de algoritmos de búsqueda

Para el Problema 3 se generaron $K = 25$ laberintos aleatorios de tamaño 45×55 . En cada escenario se eligieron posiciones inicial y final arbitrarias con distancia Manhattan mínima de 10. Para cada algoritmo se registró el número de nodos explorados, la longitud del camino y el tiempo de ejecución.

El ranking por escenario se definió según la cantidad de nodos explorados: el algoritmo con menor número de nodos explorados recibió posición 1 y el de mayor exploración recibió posición 4. Esta métrica favorece algoritmos que dirigen mejor la búsqueda hacia la meta.

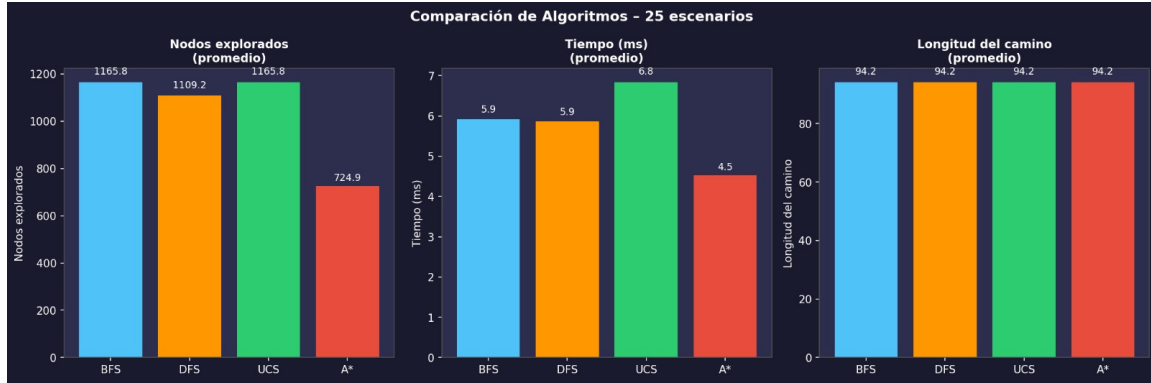


Figura 3: Promedios comparativos de nodos explorados, tiempo de ejecución y longitud de camino en 25 escenarios.

4.1. Resumen promedio

Cuadro 3: Ranking promedio sobre 25 escenarios 45×55 .

Algoritmo	Nodos promedio	Camino promedio	Tiempo promedio (ms)	Ranking promedio
A*	724.9	94.2	2.04	1.40
DFS	1109.2	94.2	2.19	2.24
BFS	1165.8	94.2	2.16	2.68
UCS	1165.8	94.2	2.96	3.68

En promedio, A* obtuvo el mejor desempeño al explorar menos nodos que los demás algoritmos. DFS quedó en segundo lugar porque en varios escenarios encontró la ruta única del laberinto con poca exploración; sin embargo, su comportamiento depende fuertemente del orden de expansión. BFS y UCS exploraron la misma cantidad de nodos en promedio, lo cual es coherente con costos uniformes. UCS resultó más lento por el uso de una cola de prioridad.

4.2. Escenarios de ejemplo

Las Figuras 4, 5 y 6 muestran tres de los escenarios generados. El repositorio también contiene visualizaciones para los 25 escenarios bajo los nombres `p3_escenario_01.png` a `p3_escenario_25.png`.

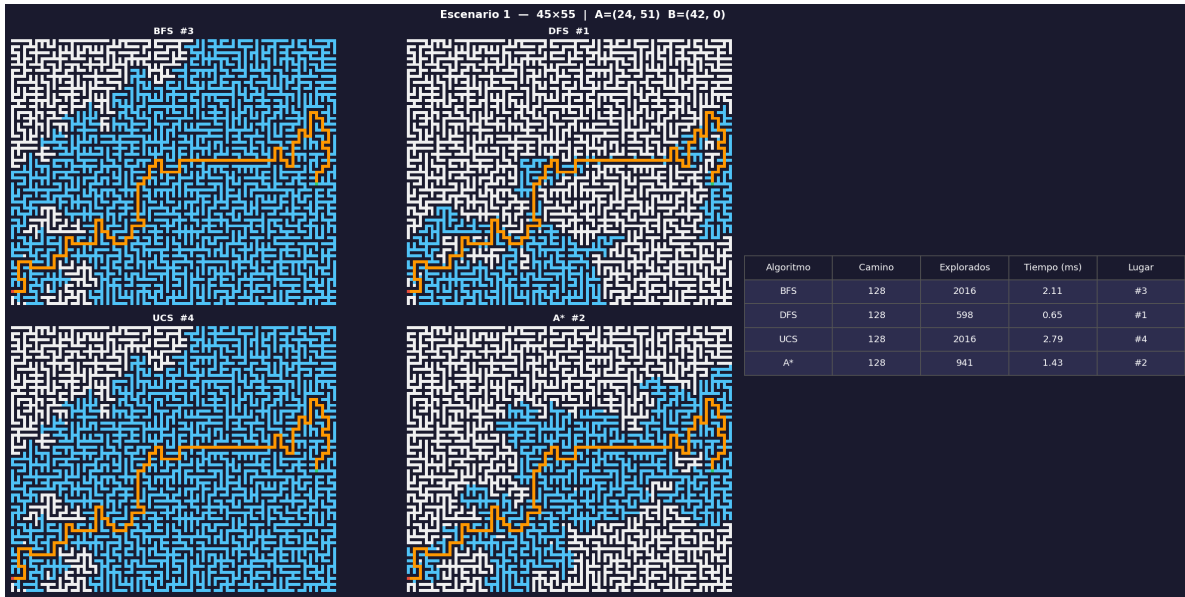


Figura 4: Escenario 1 de comparación de algoritmos.

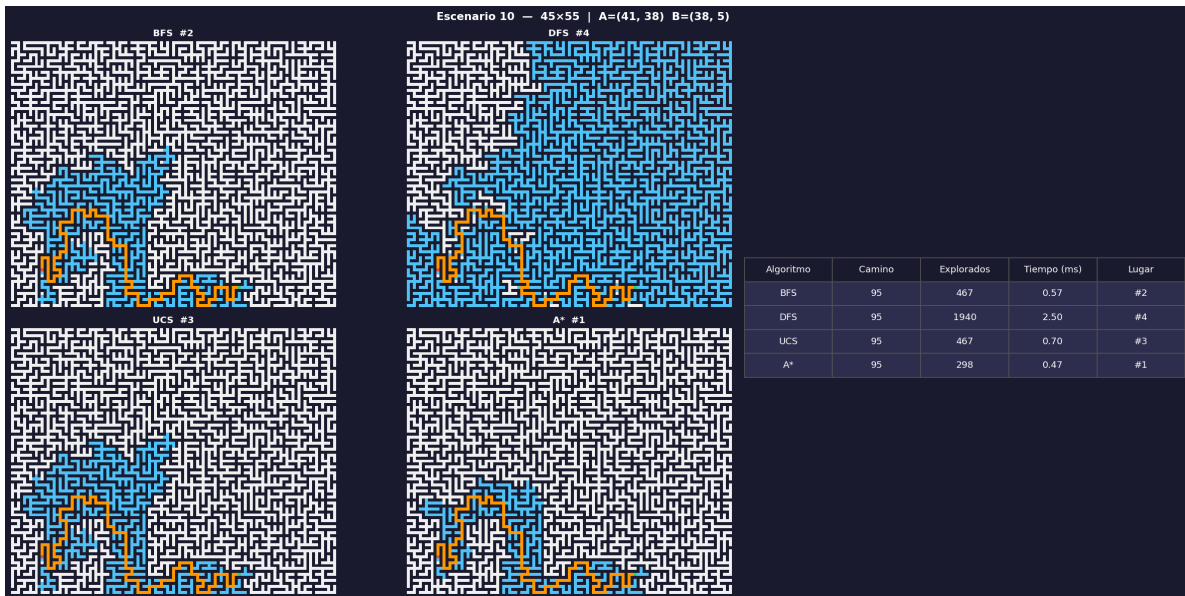


Figura 5: Escenario 10 de comparación de algoritmos.

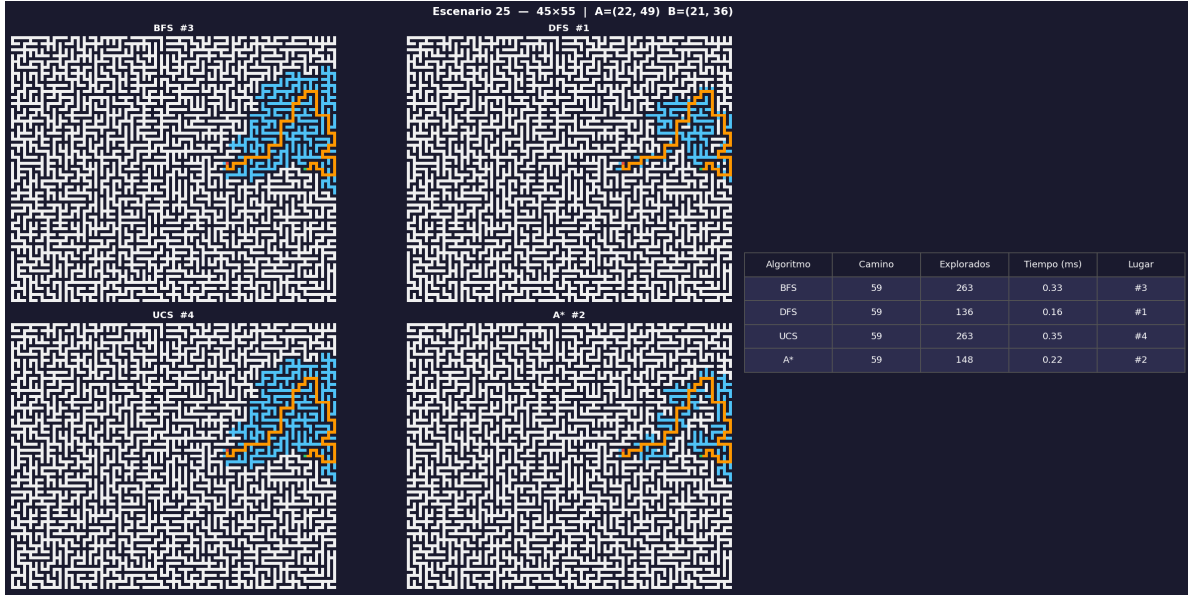


Figura 6: Escenario 25 de comparación de algoritmos.

4.3. Tabla detallada de escenarios

En la Tabla 4 se reportan los nodos explorados y el tiempo de ejecución en milisegundos para cada algoritmo. La columna “Ruta” indica la longitud del camino encontrado.

Cuadro 4: Resultados por escenario: nodos explorados/tiempo en ms.

Esc.	A	B	BFS	DFS	UCS	A*	Ruta
1	(24,51)	(42,0)	2016/3.65	598/1.17	2016/5.33	941/2.71	128
2	(19,5)	(32,39)	1084/1.99	134/0.30	1084/2.84	324/0.93	74
3	(33,36)	(4,24)	1530/2.84	1797/3.65	1530/3.92	1178/3.27	152
4	(9,8)	(38,11)	1706/3.05	1433/2.76	1706/4.40	1431/4.06	155
5	(20,21)	(43,43)	1622/2.96	1308/2.54	1622/4.14	826/2.46	102
6	(12,50)	(37,16)	1651/3.07	1139/2.23	1651/4.29	999/2.88	90
7	(17,48)	(26,29)	1258/2.33	1438/2.82	1258/3.21	731/2.05	69
8	(14,8)	(7,33)	1262/2.35	862/1.67	1262/3.16	818/2.22	115
9	(3,45)	(6,23)	577/1.06	206/0.40	577/1.43	436/1.25	58
10	(41,38)	(38,5)	467/0.89	1940/3.75	467/1.19	298/0.79	95
11	(20,46)	(22,9)	1820/3.38	1770/3.41	1820/4.53	1129/3.18	108
12	(24,42)	(15,9)	1119/2.06	2062/4.11	1119/2.84	728/2.07	97
13	(15,43)	(27,9)	1594/2.92	925/1.97	1594/4.00	836/2.35	93
14	(44,35)	(32,34)	169/0.33	2406/4.63	169/0.43	102/0.28	26
15	(1,51)	(43,3)	2070/3.74	1414/2.74	2070/5.13	1305/3.77	151
16	(22,18)	(25,29)	313/0.59	2172/4.44	313/0.77	222/0.59	61

Esc.	A	B	BFS	DFS	UCS	A*	Ruta
17	(13,35)	(22,1)	1799/3.34	328/0.67	1799/4.57	977/2.73	94
18	(25,47)	(13,9)	1587/3.04	1959/3.84	1587/3.98	1175/3.18	171
19	(37,11)	(44,4)	211/0.42	48/0.10	211/0.54	69/0.19	31
20	(38,38)	(43,19)	419/0.78	143/0.28	419/1.04	237/0.65	41
21	(12,45)	(7,6)	1532/2.86	348/0.71	1532/3.86	998/2.73	109
22	(10,25)	(31,45)	1816/3.45	1049/2.04	1816/4.55	1333/3.75	116
23	(7,51)	(38,35)	890/1.71	1845/3.60	890/2.21	573/1.58	90
24	(42,53)	(40,39)	369/0.71	270/0.53	369/0.92	309/0.84	69
25	(22,49)	(21,36)	263/0.50	136/0.27	263/0.64	148/0.40	59

5. Descripción técnica de la implementación

La arquitectura se organiza en dos archivos principales. En `maze_generator.py` se define la clase `Maze`, la estructura `UnionFind`, las funciones de generación con Kruskal y Prim, y las funciones de visualización estática y animada. El módulo permite obtener tanto el laberinto final como una lista de estados intermedios, lo cual facilita la creación de animaciones de construcción.

En `maze_solver.py` se define la clase `SearchResult`, que encapsula el camino encontrado, los nodos explorados, la longitud del camino y el tiempo de ejecución. También se implementan funciones auxiliares para obtener vecinos alcanzables, reconstruir el camino desde el diccionario de padres y pintar en la matriz las celdas exploradas, la ruta, el inicio y la meta.

Los algoritmos de búsqueda trabajan sobre las celdas lógicas del laberinto, no directamente sobre cada posición de la matriz expandida. Para determinar si un vecino es alcanzable, se revisa si la pared intermedia entre la celda actual y la celda vecina está abierta. Esta decisión reduce ambigüedades y mantiene clara la separación entre representación visual y representación lógica.

6. Conclusiones

Los algoritmos de Kruskal y Prim generaron correctamente laberintos perfectos, ya que ambos conectaron todas las celdas con exactamente $MN - 1$ aperturas de pared. En la prueba 30×40 , los dos algoritmos tuvieron tiempos muy similares, con una ligera ventaja para Prim en la corrida observada.

En el laberinto 60×80 , los cuatro algoritmos encontraron el mismo camino debido a que los laberintos generados son perfectos y, por tanto, existe una única ruta entre entrada y salida. DFS exploró menos nodos en ese caso particular; no obstante, este resultado depende del orden de vecinos y no implica que DFS sea óptimo en general.

En los 25 escenarios del Problema 3, A^* obtuvo el mejor ranking promedio. La heurística Manhattan redujo la exploración innecesaria y orientó la búsqueda hacia la meta. BFS y UCS exploraron la misma cantidad promedio de nodos porque todos los movimientos tienen costo uniforme, pero UCS fue más lento por la administración de la cola de prioridad. DFS tuvo un desempeño competitivo en varios escenarios, aunque su comportamiento es menos estable que el de A^* .

En conjunto, los resultados muestran que A^* es la alternativa más eficiente para resolver este tipo de laberintos cuando se cuenta con una heurística admisible. Para mapas con costos uniformes, BFS es una solución correcta y simple, mientras que UCS resulta más útil cuando existen pesos distintos entre movimientos o regiones del mapa.